

Performance Dynamics of Deep Learning Model Training on Centralized HPC Resources

Prashant Panta¹ and Damian Valles²

¹*Department of Computer Science, Texas State University, San Marcos, Texas, USA*

²*Ingram School of Engineering, Texas State University, San Marcos, Texas, USA*

Abstract—Short benchmark runs show hardware at its best; long runs show how it behaves. We study sustained deep learning training on the Learning, Exploration, Analysis, and Processing (LEAP2), a SLURM-managed shared university HPC cluster, using ResNet-50 trained from scratch and ViT-B/16 and BERT-base fine-tuned. We sweep three batch sizes over 100 epochs (54 runs), then train each model for 250 epochs on both 16-core Xeon Gold 6336Y CPU nodes and A100 80 GB GPUs, logging throughput, GFLOPs/s, memory, and accuracy every epoch. Four findings follow. CPU throughput declines in a workload-dependent way consistent with thermal throttling: ResNet-50 at batch size 64 loses 13 to 30% of its compute rate within a run, while every transformer run stays flat. GPU jobs submitted without exclusive node access drop to about half speed in sudden steps, the signature of a second job on the node; six of 27 sweep runs were affected. Both transformers grow host memory by about 0.4 GB over 250 epochs while the CNN does not; for ViT this costs 5% of throughput, roughly 9 hours. GPU speedup spans 7.9x to 28.3x, wide enough to change where a job should run. Logging every epoch over long runs cheaply exposes hardware behavior that short benchmarks miss.

Index Terms—HPC benchmarking, deep learning training, GPU co-tenancy, SLURM, memory characterization

I. INTRODUCTION

Most deep learning benchmarks ask how quickly hardware can train a model under controlled conditions. MLPerf Training [1] and MLPerf HPC [2] measure time to target accuracy, while DAWNBench [3] popularized time-to-accuracy benchmarking. These approaches generally assume stable, dedicated hardware. University HPC clusters are different: they are shared, scheduler-managed environments where long-running jobs may experience resource contention or changing system behavior. Although workload studies examine large GPU clusters [5], [6], they rarely characterize individual training jobs epoch by epoch.

We instead study a small number of long-running jobs with detailed instrumentation on the Learning, Exploration, Analysis, and Processing (LEAP2), the Texas State University HPC cluster managed by SLURM [4]. We train ResNet-50 [7], ViT-B/16 [8], and BERT-base [9] on CPU and GPU nodes. Phase 1 evaluates three batch sizes with three repetitions, totaling 54 100-epoch runs. Phase 2 trains each model for 250 epochs using its best batch size. Each epoch records wall time, throughput, effective GFLOPs/s, loss, accuracy, CPU RSS, and GPU memory. The models serve as workloads; the contribution is the system’s behavior revealed by detailed logging.

Our contributions are:

- We identify model- and batch-dependent CPU slowdown consistent with thermal throttling. ResNet-50 at batch size 64 loses 13–30% GFLOPs/s within 100 epochs across all repetitions, while batch size 256 and transformer workloads remain comparatively stable. The slowdown is asymmetric, with the backward pass becoming $1.80\times$ slower versus $1.08\times$ for the forward pass.
- We observe GPU throughput interference consistent with node sharing. Jobs submitted without `--exclusive` experience abrupt reductions to approximately half throughput, affecting six of 27 GPU sweep runs. Higher GPU memory utilization reduces but does not eliminate this behavior.
- We find approximately 0.4 GB of CPU RSS growth over 250 epochs for both ViT and BERT, but not ResNet-50 or GPU memory. For ViT, this growth coincides with a 5.1% increase in epoch time, adding approximately nine hours to the run despite normal loss and accuracy.
- We quantify model-dependent A100 acceleration: $28.3\times$ for ResNet-50, $18.3\times$ for BERT-base, and $7.9\times$ for ViT-B/16, demonstrating that GPU benefit varies substantially across workload families and should inform job placement on shared clusters.

II. RELATED WORK

Deep learning benchmarking has largely focused on performance under controlled or dedicated hardware conditions. MLPerf Training [1], [10] measures time-to-accuracy across standardized models, while MLPerf HPC [2] extends this approach to scientific workloads. DAWNBench [3] introduced time- and cost-to-accuracy metrics, DeepBench [11] evaluates individual kernels, and ParaDnn [12] compares TPU, GPU, and CPU architectures. These studies generally emphasize relatively short runs, whereas our work examines how training behavior changes from epoch 1 through epoch 250 on a shared HPC cluster.

Long-duration performance can also be affected by thermal behavior, scheduling, and resource sharing. Prior DVFS studies have demonstrated substantial thermal throttling on edge platforms [20]–[22], while server-class processors are generally expected to maintain stable performance. Our results instead reveal repeatable, workload-dependent CPU slowdown consistent with throttling. Shared GPU clusters have been characterized at scale [5], [6], and schedulers such as Gandiva [23]

TABLE I
SYSTEM CONFIGURATION

	CPU jobs	GPU jobs
Partition	parallel	gpu1
Processor	2x Xeon Gold 6336Y	A100 80 GB PCIe
Cores / job	16	4 (host) + 1 GPU
RAM / job	64 GB	32 GB (host)
Exclusive	Phase 2 (flag); Phase 1 in effect	Phase 2 only
Threads	OMP/MKL = 16	OMP/MKL = 4
CUDA	n/a	11.8
Framework	PyTorch + HuggingFace	Transformers

and Tiresias [24] deliberately manage sharing and migration [25]. Hardware mechanisms such as NVIDIA MIG have also been evaluated for controlled sharing [26]–[28]. In contrast, we examine unintended node sharing caused by non-exclusive scheduler allocation and characterize the resulting slowdown from within affected training jobs.

Memory and energy studies provide additional context for long-running workloads. Existing memory research primarily targets GPU activation, data movement, and checkpointing [29]–[31], leaving long-term host-memory behavior comparatively unexplored. We measure CPU memory across 250 epochs and identify growth associated with transformer workloads. Energy-aware approaches such as Zeus [33] and GPU power-capping studies [34], [35] optimize training efficiency through batch size and power management. Although we do not directly measure energy, our observed CPU slowdown, model-dependent GPU speedups, and per-epoch GFLOPs/s measurements expose system behavior relevant to these optimization strategies.

III. SYSTEM CONFIGURATION

LEAP2 is the production HPC cluster at Texas State University, managed by SLURM [4]. CPU jobs ran on the `parallel` partition using nodes with two Intel Xeon Gold 6336Y (Ice Lake) processors. Each job requested 16 cores and 64 GB RAM, with `OMP_NUM_THREADS=16`, `MKL_NUM_THREADS=16`, and PyTorch configured for 16 threads. Phase 2 CPU jobs used `--exclusive`, while Phase 1 jobs effectively occupied their allocated resources. GPU jobs ran on the `gpu1` partition using NVIDIA A100 80 GB PCIe GPUs [19] with CUDA 11.8, 4 host cores, and 32 GB host RAM. Phase 2 GPU jobs used `--exclusive`, whereas Phase 1 jobs did not, allowing scheduler-induced node sharing that produced the interference examined in Section VI-C. All experiments used Python 3, PyTorch [14], and HuggingFace Transformers [15] within a common conda environment, so CPU and GPU runs differed only in device and thread configuration. Tokenizer parallelism was disabled using `TOKENIZERS_PARALLELISM=false` to avoid fork overhead during BERT data loading. Table I summarizes both CPU and GPU configurations.

TABLE II
MODEL AND DATASET SUMMARY

	ResNet-50	ViT-B/16	BERT-base
Architecture	CNN	Vision transformer	Language transformer
Parameters	23.5M	85.8M	109.5M
Dataset	CIFAR-10	CIFAR-10	IMDB
Training samples	50,000	50,000	25,000
Training mode	From scratch	Fine-tune	Fine-tune
Phase 2 batch size	256	128	256

IV. METHODOLOGY

A. Models and Datasets

We picked one model from each of three families so that differences in hardware behavior come from the workload, not from unrelated setup choices. ResNet-50 [7] (23.5M parameters) is a convolutional network trained from scratch on CIFAR-10 [16], with images resized to 224x224; it starts near the 10% random-guess level. ViT-B/16 [8] (85.8M parameters) is a vision transformer fine-tuned on CIFAR-10 from pre-trained ImageNet weights, starting at about 97.9% validation accuracy. BERT-base [9] (109.5M parameters) is a language transformer fine-tuned on IMDB sentiment classification [17] with 25,000 training reviews, a 5,000-review validation set, and text cut or padded to 128 tokens. ResNet-50 uses SGD (momentum 0.9, learning rate 0.1, weight decay 5×10^{-4}); both transformers use AdamW (10^{-4} for ViT, 2×10^{-5} for BERT). Table II summarizes the three.

B. Experimental Phases

The experiments were conducted in two phases. Phase 1 trained each model for 100 epochs at batch sizes 64, 128, and 256, with three repetitions per configuration and a fixed learning rate, producing 27 CPU and 27 GPU runs (54 total). The sweep established throughput baselines, identified system-level performance issues, and selected batch sizes for Phase 2. ResNet-50 used batch size 256, which eliminated the throughput degradation observed at 64 (Section V-B). BERT-base also used 256, providing similar throughput to smaller batches with lower host-memory use. ViT-B/16 used 128 because batch size 256 provided negligible throughput improvement (approximately 155 vs. 154 images/s) while increasing GPU memory from 16.2 to 31.2 GB. Phase 1 GPU jobs were submitted without `--exclusive`, allowing the co-tenancy effects examined later.

Phase 2 trained each model for 250 epochs using CosineAnnealingLR [18] with $T_{max} = 250$. CPU training used a chain of five 50-epoch SLURM jobs, with checkpoints preserving model, optimizer, and scheduler state between jobs and a SIGTERM handler providing emergency checkpointing. Wall-times were set to 50 hours for ResNet-50 and BERT-base chunks and 80 hours for ViT-B/16. This structure introduces brief idle periods between CPU jobs. GPU training instead used one 250-epoch job per model with a 120-hour walltime and `--exclusive`. GPU job wall time therefore includes startup, dataset loading, CUDA initialization, checkpointing, and plotting overhead. For example, ResNet-50 accumulated

3.54 hours of per-epoch training within a 3.79-hour SLURM job. Accordingly, GPU wall time refers to total SLURM job duration, whereas CPU training time is reported as the sum of per-epoch times across the five-job chain.

C. Performance Proxy and Metrics

Each epoch computes an effective GFLOPs/s metric from the estimated floating-point work per sample (8.2 GFLOPs for ResNet-50, 52.8 for ViT-B/16, and 67.5 per 128-token BERT-base sequence, including forward and backward passes), the number of samples processed, and measured epoch time. For a fixed model, batch size, and dataset, computational work per epoch is constant; therefore, changes in effective GFLOPs/s indicate changes in system performance rather than workload. This is particularly useful on CPU nodes where users lack access to hardware counters or privileged monitoring tools. We use the metric as a proxy for detecting performance degradation consistent with DVFS-based thermal throttling. Although its absolute value depends on the FLOP estimate, its epoch-to-epoch trend does not.

Validation checkpoints are saved every 5 epochs through epoch 30, every 10 through epoch 100, and every 25 through epoch 250, concentrating measurements where learning changes most rapidly while reducing storage and I/O overhead later. Every epoch records wall time, throughput (images/s or tokens/s), effective GFLOPs/s, training and validation loss and accuracy, CPU memory using `psutil`, GPU memory using `torch.cuda.max_memory_allocated`, and time spent in data loading, forward pass, backward pass, and optimizer steps. GPU timing uses `torch.cuda.synchronize` for accurate epoch measurements, although per-phase GPU percentages remain approximate because asynchronous kernel execution can overlap phase boundaries.

V. RESULTS: CPU TRAINING

A. Convergence Behavior

Figures 1 and 2 show the ResNet-50 and ViT-B/16 CPU runs. ResNet-50 validation accuracy rises from 11.4% to 95.43% while training loss falls from 3.09 to 0.002, with no instability across 99.9 hours. Mean epoch time is 1439.2 s with a standard deviation of only 4.1 s (0.3%), making this run the study’s stability control. ViT-B/16 begins near 97.9% validation accuracy from pretrained ImageNet weights and peaks at 98.12% at epoch 124. Its 178.6-hour run is the longest in the study and shows greater epoch-time variation (2571.8 ± 66.6 s), which Section V-C associates with host-memory growth rather than changes in learning behavior.

Figure 3 shows BERT-base, where validation accuracy rises from 88.1% to 93.60% at epoch 74 before fluctuating as training accuracy approaches 100%, consistent with overfitting during intentionally extended fine-tuning. Despite running for 131.9 hours, BERT remains computationally stable, with a mean epoch time of 1898.9 s and a standard deviation of 10.4 s. Thus, while the three models exhibit different learning behavior, their long CPU runs provide stable baselines for

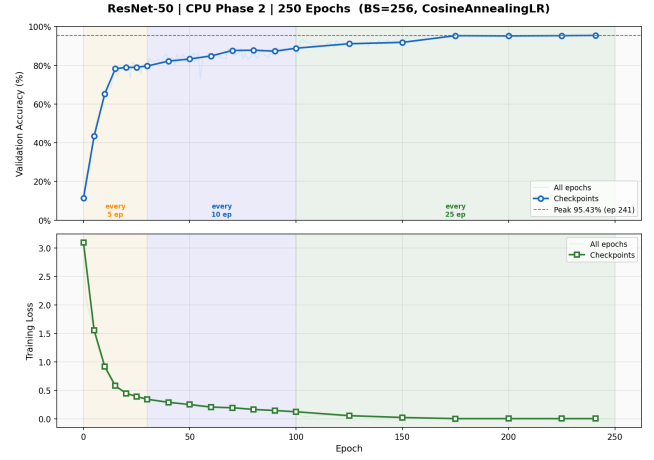


Fig. 1. ResNet-50 CPU Phase 2 over 250 epochs at batch size 256. Top: validation accuracy with checkpoint markers over the full per-epoch trace. Bottom: training loss. Shaded zones mark the checkpoint sampling density (orange: every 5 epochs; blue: every 10; green: every 25).

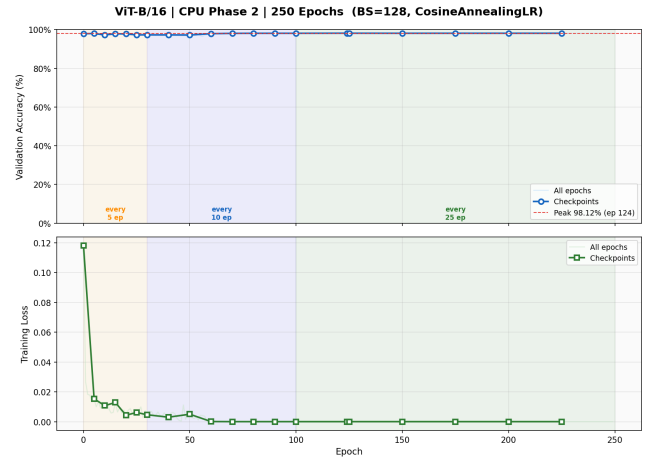


Fig. 2. ViT-B/16 CPU Phase 2 over 250 epochs at batch size 128. The pretrained model enters near its final accuracy; the interesting behavior in this run is in throughput and memory (Fig. 6), not in the learning curves.

distinguishing training dynamics from the system-level performance effects examined later.

B. Workload-Dependent Throughput Degradation on CPU Nodes

The Phase 1 sweep revealed the study’s clearest hardware effect. ResNet-50 at batch size 64 lost 26.6%, 30.2%, and 13.2% GFLOPs/s across three repeats, with epoch times increasing from approximately 1360 s to as much as 1950 s. The gradual slowdown repeated across separate jobs, while increasing batch size reduced the effect: losses fell to 2.1–11.5% at batch size 128 and below 1% at 256. ViT and BERT remained within approximately 3% at all batch sizes. Timing logs further localized the slowdown to the backward pass. At batch size 64, backward-pass time increased by up to $1.80\times$, compared with only $1.08\times$ for the forward pass, while data-loading time remained stable. This workload-dependent,

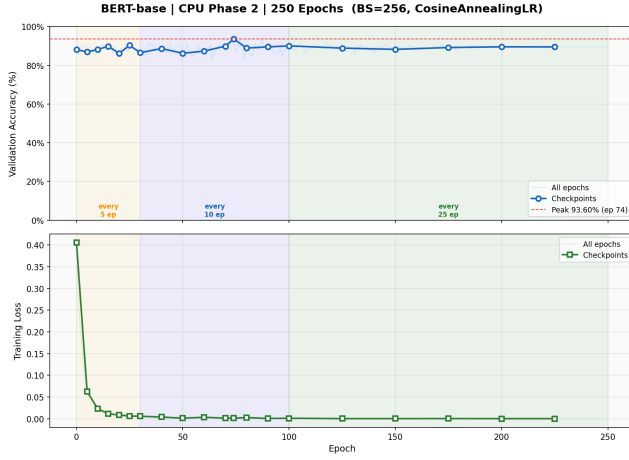


Fig. 3. BERT-base CPU Phase 2 over 250 epochs at batch size 256. Validation accuracy peaks early (93.60% at epoch 74) and then fluctuates while training accuracy saturates, the expected outcome of fine-tuning far past the recommended epoch budget.

gradual behavior is consistent with DVFS thermal throttling rather than a system-wide software or I/O event, although temperature and frequency telemetry would be required to confirm the cause. Figure 4 shows the GFLOPs/s trajectories that characterize this effect.

Phase 2 demonstrates the stabilized configuration. ResNet-50 at batch size 256 drifted by less than 0.1% in GFLOPs/s across 250 epochs, likely benefiting from both the larger batch size and idle cooling periods between 50-epoch CPU jobs. The results show that batch size can influence sustained CPU performance beyond its usual role in training efficiency. More importantly for benchmarking, throughput measured during early epochs on a cold node can substantially overestimate long-run performance, by as much as 30% for the workloads observed here.

C. GFLOPs/s Trajectory Under Sustained Thermal Load

Figure 4 shows effective GFLOPs/s across the 100 Phase 1 epochs for three ResNet-50 batch-size-64 runs, with the batch-size-256 mean as a reference. All small-batch runs begin near cold-node peak throughput before gradually declining by 26.6%, 30.2%, and 13.2%. The smooth, monotonic decline differs from the abrupt changes expected from scheduler or I/O events. In contrast, batch size 256 remains within 0.1% of its initial GFLOPs/s throughout the run.

The divergence is consistent with workload-dependent thermal accumulation. At batch size 64, longer processing time per sample keeps CPU arithmetic units active longer, potentially increasing junction temperature and triggering DVFS. Batch size 256 processes more work per second and may benefit from greater computational efficiency and brief inter-epoch cooling. Although hardware temperature and frequency telemetry are required to confirm the mechanism, the practical result is clear: throughput measured during the first 10 epochs on a cold CPU node can overestimate sustained long-run performance by up to 30% for this workload.

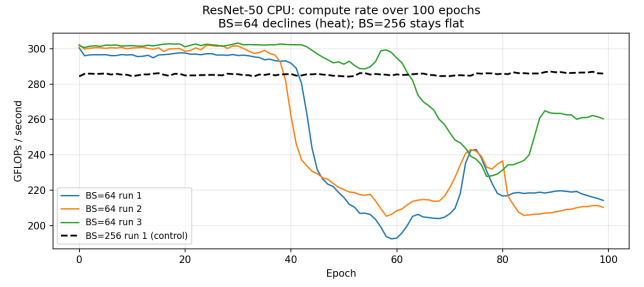


Fig. 4. Effective GFLOPs/s over 100 Phase 1 epochs for ResNet-50 on CPU. All three batch-size-64 repetitions (solid lines) decline gradually from their cold-node starting values, with losses of 13–30% by epoch 100. Batch size 256 (dashed) remains flat throughout, confirming that the degradation is batch-size-dependent and consistent with DVFS thermal throttling.

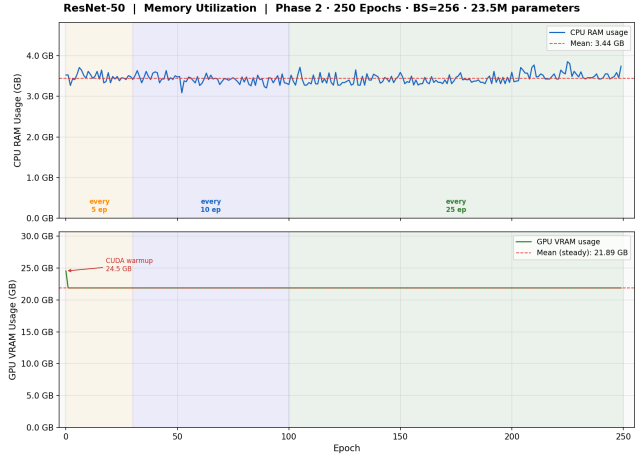


Fig. 5. ResNet-50 memory over 250 epochs. Top: CPU RSS. Bottom: GPU VRAM. RSS fluctuates around a stable 3.44 GB mean with no sustained trend; VRAM settles at 21.89 GB after a 24.54 GB allocator peak in the warmup epoch.

D. Host Memory Growth in Transformer Training

Figures 5, 6, and 7 show per-epoch memory behavior, summarized in Table III. ResNet-50 shows no sustained CPU-memory growth, remaining between 3.09 and 3.85 GB with a 3.44 GB mean. In contrast, both transformer models accumulate host memory. BERT-base grows from approximately 3.48 to 3.88 GB, including increases within individual 50-epoch processes. ViT-B/16 rises from approximately 2.63 to 3.00 GB through both checkpoint-related increases between job chunks and gradual 0.1–0.16 GB growth within each process. Because each chunk starts a new Python process, the within-chunk growth develops during training rather than persisting across process restarts. The behavior appears in both HuggingFace transformer workloads but not ResNet-50, while GPU memory remains stable, indicating a host-side effect associated with the transformer software stack.

The performance impact differs by model. BERT’s approximately 0.40 GB increase is small relative to its 64 GB allocation and does not noticeably affect epoch time. However, ViT loses 4.9% GFLOPs/s between its early and late epochs,

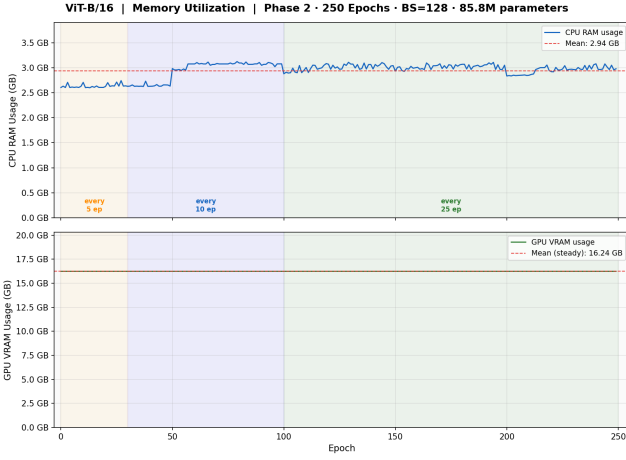


Fig. 6. ViT-B/16 memory over 250 epochs. RSS steps from 2.63 GB to 2.98 GB at the epoch 50 chunk boundary and continues a slow climb within chunks; VRAM holds at 16.24 GB with under 1 MB of epoch-to-epoch variation.

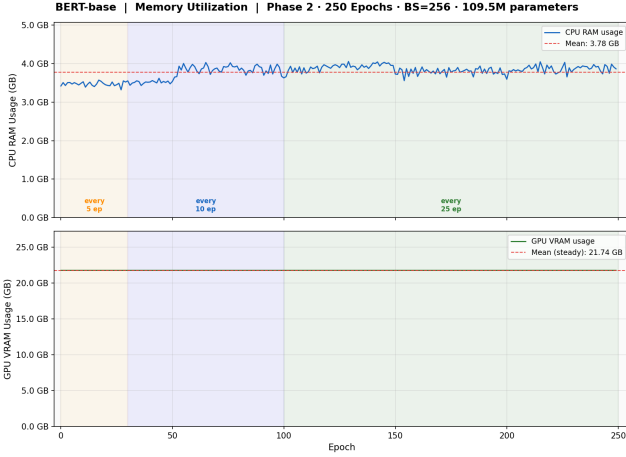


Fig. 7. BERT-base memory over 250 epochs. RSS grows from 3.48 GB to 3.88 GB with visible within-chunk accumulation; VRAM holds at 21.74 GB with under 10 MB of variation.

TABLE III
MEMORY BEHAVIOR SUMMARY (PHASE 2)

Model	RSS mean	RSS drift	VRAM warmup	VRAM steady
ResNet-50	3.44 GB	none (osc. ± 0.2 GB)	24.54 GB	21.89 GB
ViT-B/16	2.94 GB	+0.38 GB	no spike	16.24 GB
BERT-base	3.78 GB	+0.40 GB	no spike	21.74 GB

while epoch time increases from 2527 to 2657 s, a 5.1% slowdown that coincides with memory growth. Across the 178.6-hour run, this difference corresponds to approximately nine additional hours. Thus, modest host-memory growth can affect long-running transformer performance even when GPU memory, loss, and accuracy curves appear normal.

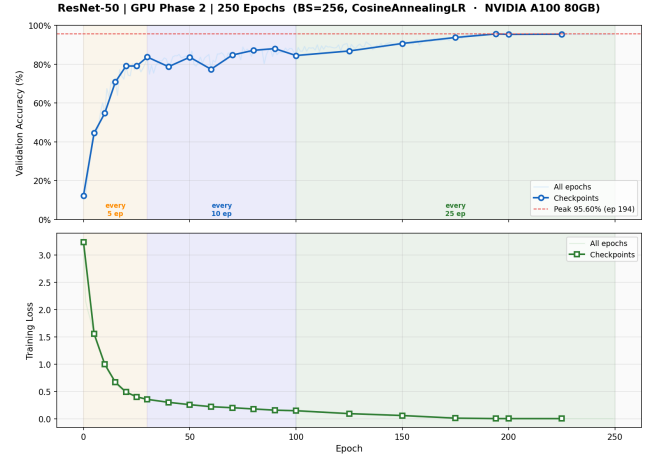


Fig. 8. ResNet-50 GPU Phase 2 over 250 epochs at batch size 256 on an A100 80 GB. The learning trajectory matches the CPU run epoch for epoch; the entire 250-epoch job completes in 3.79 hours.

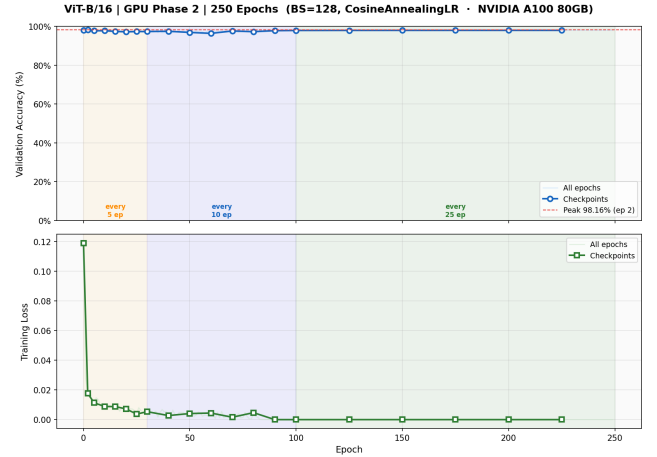


Fig. 9. ViT-B/16 GPU Phase 2 over 250 epochs at batch size 128. Peak validation accuracy (98.16%) occurs at epoch 2, a property of fine-tuning from strong pretrained weights rather than an anomaly.

VI. RESULTS: GPU TRAINING

A. Convergence Behavior

Figure 8 shows ResNet-50 on the A100. The learning curve matches the CPU run: accuracy rises from the random floor to a peak of 95.60% at epoch 194, within 0.17 points of the CPU peak, so the device changed the speed but not the learning. The contrast is the point. Mean epoch time is 50.9 s with a 0.7 s standard deviation, and all 250 epochs finish in a 3.79-hour job, against 99.9 hours on CPU for the same curve. With exclusive allocation, the GPU run has none of the Phase 1 instability: throughput is flat from the second epoch on.

Figure 9 shows ViT-B/16. Its best validation accuracy, 98.16%, comes at epoch 2: the pretrained ImageNet features carry over to CIFAR-10 almost fully, so the model starts near convergence, hits its peak in the first few epochs, and settles a little lower as the optimizer keeps moving. Pretrained models peaking early is expected and needs no more comment. The

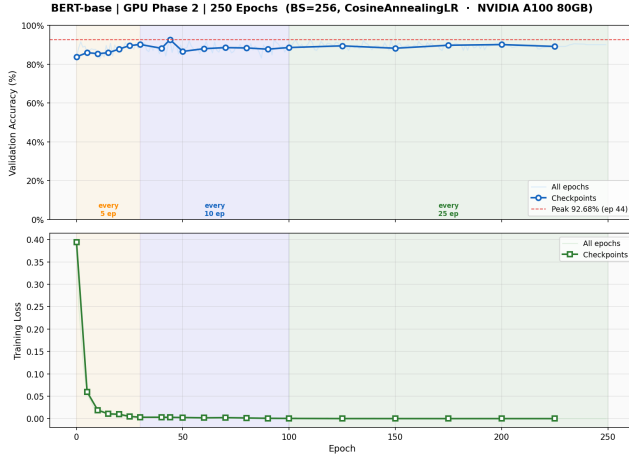


Fig. 10. BERT-base GPU Phase 2 over 250 epochs at batch size 256. Validation accuracy peaks at 92.68% (epoch 44) and fluctuates thereafter while training loss collapses, mirroring the CPU run’s overfitting profile at 18.3x the speed.

hardware story is steadiness: 324.9 s mean epoch time with a 1.2 s standard deviation over 24.1 hours, and the GFLOPs/s estimate even rises 0.6% over the run. The host memory growth that cost the CPU run 5% of its throughput has no measurable effect here, because the GPU run’s bottleneck is on the device.

Figure 10 shows BERT-base, which peaks at 92.68% at epoch 44 and then swings, the same overfitting as on CPU but packed into 7.70 hours. Its epoch time is the steadiest measurement in the study: 104.0 s mean with a 0.16 s standard deviation, a spread of only 0.15%. Long A100 training under exclusive allocation is, in practice, steady in time; every throughput oddity we saw on GPUs came from the scheduler, not the chip.

B. VRAM Behavior

The bottom panels of Figures 5 through 7 show GPU memory. ResNet-50 hits a 24.54 GB peak in the warmup epoch, when PyTorch’s CUDA allocator sizes its pools and cuDNN picks convolution algorithms, then settles to 21.89 GB and stays within 0.3 GB per epoch for the other 249 epochs. Neither transformer has a warmup spike: ViT holds 16.24 GB with under 1 MB of change, and BERT holds 21.74 GB with under 10 MB. Once the allocator settles, GPU memory is basically fixed for a given setup. The contrast with the host side is sharp: the same 250 epochs that grow transformer host memory by 0.4 GB leave GPU memory rock-steady, which places the buildup in host-side runtime, not in anything the models put on the device.

C. Implicit Co-Tenancy in Phase 1

Phase 1 GPU jobs were submitted without `--exclusive`, allowing SLURM on LEAP2 to place multiple jobs on available nodes. Per-epoch logging revealed interference in six of 27 sweep runs (Table IV). For ResNet-50 at batch size 64, one run remained near 937 images/s, another began at roughly half

speed before recovering near epoch 80, and a third abruptly dropped from approximately 916 to 450 images/s at epoch 68 before recovering. Similar behavior appeared for BERT at batch sizes 64 and 128 and ViT at 256. These sudden transitions, unlike the gradual CPU slowdown in Section V-B, are consistent with jobs entering or leaving the node. No interference occurred across the 750 Phase 2 GPU epochs executed with `--exclusive`.

Higher GPU memory use reduced interference but did not eliminate it. ResNet-50 at batch size 64 used 5.7 GB and experienced the most interference, while ViT at 31.2 GB was still affected twice. Thus, memory utilization alone does not prevent co-tenancy. Unlike deliberate GPU-sharing approaches such as NVIDIA MIG [26], [27] and co-location schedulers [25], [28], this sharing was unintended and invisible to users, making per-epoch throughput logging useful for detecting otherwise unexplained performance loss. Figure 11 shows the per-epoch throughput trajectories that expose this behavior for ResNet-50 at batch size 64.

D. Per-Epoch Throughput Signatures of Co-Tenancy

Figure 11 shows epoch-level throughput for the three ResNet-50 GPU batch-size-64 Phase 1 runs, revealing three co-tenancy scenarios. Run 1 remains near 937 images/s throughout, consistent with an uncontested node. Run 2 begins at roughly half speed before abruptly recovering near epoch 80, consistent with a co-located job present at submission and later completing. Run 3 shows the reverse behavior, maintaining full throughput until epoch 68 before suddenly dropping to approximately half speed, consistent with another job arriving during training.

These trajectories demonstrate the uncertainty of non-exclusive GPU submissions. An uncontested job can perform like an exclusive run, while jobs sharing a node may lose roughly half their throughput without an explicit warning. Importantly, the step-function behavior differs from the gradual CPU slowdown in Figure 4: co-tenancy produces abrupt throughput transitions, whereas thermal effects produce sustained slopes. Per-epoch logging therefore provides a practical way to distinguish these otherwise hidden system-level performance effects.

VII. CROSS-PLATFORM COMPARISON

A. Architecture-Dependent Speedup

Table V summarizes Phase 2, showing a $3.6\times$ spread in A100 speedup across identical hardware and software configurations, from $7.9\times$ for ViT-B/16 to $28.3\times$ for ResNet-50, with BERT-base at $18.3\times$. This variation follows arithmetic intensity [32]: ResNet-50’s dense, regular convolutions efficiently use A100 tensor cores, while ViT spends more time in attention operations with greater memory traffic relative to computation [13], [30]. BERT falls between these workloads through its combination of attention and feed-forward layers. Interestingly, ViT achieves the highest CPU GFLOPs/s (approximately 1050 versus 285 for ResNet-50), reflecting the efficiency of large matrix operations on MKL. Thus, the

TABLE IV
PHASE 1 GPU THROUGHPUT BY BATCH SIZE (MEAN PER RUN; IMG/S FOR RESNET-50 AND ViT, TOKENS/S FOR BERT). DAGGERS MARK RUNS CONTAMINATED BY CO-TENANCY.

Model	BS	Run 1	Run 2	Run 3	Steady VRAM	Affected runs
ResNet-50	64	937	548 [†]	798 [†]	5.7 GB	2 of 3
ResNet-50	128	967	966	965	11.1 GB	0
ResNet-50	256	981	981	981	21.9 GB	0
ViT-B/16	64	151	149	152	8.8 GB	0
ViT-B/16	128	154	154	151	16.2 GB	0
ViT-B/16	256	140 [†]	147 [†]	156	31.2 GB	2 of 3
BERT-base	64	23,303 [†]	27,820	27,841	6.8 GB	1 of 3
BERT-base	128	29,177	24,197 [†]	29,181	11.8 GB	1 of 3
BERT-base	256	29,865	29,863	29,864	21.8 GB	0

TABLE V
PHASE 2 PERFORMANCE SUMMARY (250 EPOCHS PER RUN; CPU TIME IS THE SUM OF PER-EPOCH TIMES ACROSS THE JOB CHAIN, GPU TIME IS SLURM WALL TIME)

Model	CPU epoch (s)	GPU epoch (s)	CPU throughput	GPU throughput	Speedup	CPU time (h)	GPU time (h)	Peak val (CPU)	Peak val (GPU)
ResNet-50	1439.2 $\pm$ 4.1	50.9 $\pm$ 0.7	34.7 img/s	981.6 img/s	28.3x	99.9	3.79	95.43% (ep 241)	95.60% (ep 194)
ViT-B/16	2571.8 $\pm$ 66.6	324.9 $\pm$ 1.2	19.5 img/s	153.9 img/s	7.9x	178.6	24.06	98.12% (ep 124)	98.16% (ep 2)
BERT-base	1898.9 $\pm$ 10.4	104.0 $\pm$ 0.2	1,629 tok/s	29,744 tok/s	18.3x	131.9	7.70	93.60% (ep 74)	92.68% (ep 44)

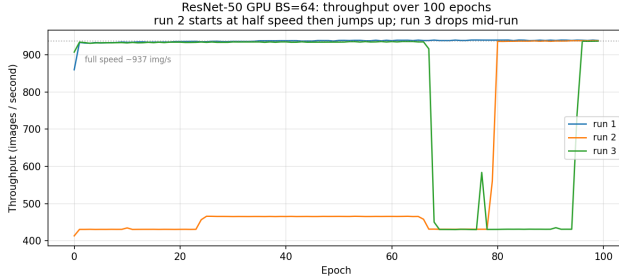


Fig. 11. Per-epoch GPU throughput for ResNet-50 at batch size 64 across three Phase 1 repetitions. Run 1 is unaffected. Run 2 begins at half speed and recovers near epoch 80 when the co-located job exits. Run 3 runs at full speed before an abrupt drop at epoch 68 when a second job arrives on the node. The step-function transitions distinguish co-tenancy from the gradual slope of CPU thermal throttling (Fig. 4).

GPU does not make ViT inherently slow; rather, it accelerates ResNet-50 exceptionally well, demonstrating that speedup is a property of the model–hardware combination rather than the model alone.

B. Accuracy Consistency

Peak validation accuracy matches across platforms within normal run-to-run noise: 95.43 vs 95.60% for ResNet-50, 98.12 vs 98.16% for ViT, and 93.60 vs 92.68% for BERT, where the CPU run’s higher peak is just one sample from BERT’s wide post-overfitting swings, not a platform effect. So, none of our findings are muddled by the two platforms training different models.

C. Device Memory Versus Host Memory

The two kinds of memory behave in opposite ways over long runs. GPU memory, after warmup, is flat: within 0.3 GB for ResNet-50 and within megabytes for the transformers

across 250 epochs. Host memory is not: flat for the CNN, but always rising for both transformers over our runs. Someone sizing memory from a short test will get GPU memory exactly right and transformer host memory more and more wrong as the run goes on. The growth here is small, but it is steady, never reclaimed, and invisible to accuracy, and for ViT it costs measurable throughput (Section V-D).

VIII. DISCUSSION

Our results highlight several practical issues for deep learning on shared HPC systems. Throughput without a time axis can be misleading: ResNet-50 at batch size 64 begins near 300 GFLOPs/s on a cold LEAP2 CPU node but sustains only 210–260 GFLOPs/s over longer runs, showing that batch size affects sustained CPU performance as well as training efficiency. Per-epoch logging and counter-free GFLOPs/s estimates can detect these changes without hardware counters. Resource allocation also matters: six of 27 Phase 1 GPU runs experienced co-tenancy without `--exclusive`, reducing throughput by roughly half. Sites could use exclusive allocation, expose sharing in job accounting, or provide controlled sharing through MIG [26]. Long-running transformer workloads also showed host-memory growth while GPU memory remained stable; ViT lost approximately 5.1% throughput over 178 hours, adding roughly nine hours of training. Monitoring host memory and sustained performance is therefore important for long jobs.

The GPU benefits also varied substantially by workload. Moving ResNet-50 from CPU to GPU saved approximately 96 hours, while ViT saved 154 hours but occupied the GPU 6.3 times longer and achieved only a 7.9 $\times$ speedup. Expected acceleration and arithmetic intensity could therefore help schedulers place workloads when GPU capacity is limited. Energy-aware methods such as Zeus [33] and power capping [34], [35] could further improve efficiency. Limitations include

evaluation on a single cluster, one Phase 2 run per configuration, lack of hardware telemetry, indirect identification of co-tenancy, and unresolved causes of transformer memory growth. Future work will validate these effects using additional systems, hardware telemetry, scheduler records, and detailed memory profiling.

IX. CONCLUSION

We logged 60 training runs on a shared university HPC cluster epoch by epoch, revealing five system-level behaviors that short benchmarks can miss. CPU training showed workload-dependent slowdown consistent with thermal throttling, with small-batch convolutional workloads losing 13–30% of their compute rate while larger batches and transformer workloads remained stable; a counter-free GFLOPs/s estimate was sufficient to detect the effect. GPU jobs submitted without `--exclusive` experienced sudden half-speed drops consistent with node sharing, which per-epoch logging exposed but total wall time concealed. Transformer workloads accumulated approximately 0.4 GB of host memory over 250 epochs, with ViT losing 5.1% throughput and roughly nine hours of runtime despite normal accuracy curves. GPU speedup varied substantially across models, from $7.9\times$ to $28.3\times$, suggesting that workload characteristics should inform job placement. Finally, expected overfitting during 250-epoch fine-tuning confirmed that the workloads remained computationally healthy. Future work will incorporate node-level power and temperature telemetry, controlled co-tenancy experiments, detailed transformer memory profiling, direct energy measurements, and additional model families to validate and extend these findings.

ACKNOWLEDGMENT

We thank the Texas State University Division of Information Technology for operating the LEAP2 cluster and supporting this work. The authors used Claude (Anthropic) and Grammarly to assist in code development, figure generation, and editing language in the manuscript text drafted by the authors; all analyses, results, and conclusions were generated, reviewed, and verified by the authors.

REFERENCES

- [1] P. Mattson et al., “MLPerf training benchmark,” in *Proc. Machine Learning and Systems (MLSys)*, 2020.
- [2] S. Farrell et al., “MLPerf HPC: A holistic benchmark suite for scientific machine learning on HPC systems,” in *Proc. IEEE/ACM Workshop on Machine Learning in High Performance Computing Environments (MLHPC)*, 2021.
- [3] C. Coleman et al., “Analysis of DAWNbench, a time-to-accuracy machine learning performance benchmark,” *ACM SIGOPS Operating Systems Review*, vol. 53, no. 1, pp. 14–25, 2019.
- [4] A. B. Yoo, M. A. Jette, and M. Grondona, “SLURM: Simple Linux utility for resource management,” in *Proc. Job Scheduling Strategies for Parallel Processing (JSSPP)*, 2003.
- [5] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, “Analysis of large-scale multi-tenant GPU clusters for DNN training workloads,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2019.
- [6] Q. Hu, P. Sun, S. Yan, Y. Wen, and T. Zhang, “Characterization and prediction of deep learning workloads in large-scale GPU datacenters,” in *Proc. Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [8] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2021.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019.
- [10] P. Mattson et al., “MLPerf: An industry standard benchmark suite for machine learning performance,” *IEEE Micro*, vol. 40, no. 2, pp. 8–16, 2020.
- [11] Baidu Research, “DeepBench: Benchmarking deep learning operations on different hardware,” 2016. [Online]. Available: <https://github.com/baidu-research/DeepBench>
- [12] Y. E. Wang, G.-Y. Wei, and D. Brooks, “Benchmarking TPU, GPU, and CPU platforms for deep learning,” arXiv:1907.10701, 2019.
- [13] A. Vaswani et al., “Attention is all you need,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [14] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [15] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP: System Demonstrations*, 2020.
- [16] A. Krizhevsky, “Learning multiple layers of features from tiny images,” Univ. of Toronto, Tech. Rep., 2009.
- [17] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, “Learning word vectors for sentiment analysis,” in *Proc. 49th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2011.
- [18] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [19] NVIDIA Corporation, “NVIDIA A100 tensor core GPU architecture,” Whitepaper, 2020.
- [20] T. Benoit-Cattin, D. Velasco-Montero, and J. Fernández-Berni, “Impact of thermal throttling on long-term visual inference in a CPU-based edge device,” *Electronics*, vol. 9, no. 12, art. 2106, 2020.
- [21] Y. Zhou, F. Liang, T.-W. Chin, and D. Marculescu, “Play it cool: Dynamic shifting prevents thermal throttling,” in *ICML Workshop on Dynamic Neural Networks*, 2022, arXiv:2206.10849.
- [22] S. Kim, K. Bin, S. Ha, K. Lee, and S. Chong, “zTT: Learning-based DVFS with zero thermal throttling for mobile devices,” in *Proc. 19th Int. Conf. on Mobile Systems, Applications, and Services (MobiSys)*, 2021.
- [23] W. Xiao et al., “Gandiva: Introspective cluster scheduling for deep learning,” in *Proc. 13th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2018.
- [24] J. Gu et al., “Tiresias: A GPU cluster manager for distributed deep learning,” in *Proc. 16th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2019.
- [25] Z. Ye et al., “Deep learning workload scheduling in GPU datacenters: A survey,” *ACM Computing Surveys*, vol. 56, no. 6, art. 146, 2024.
- [26] B. Li, V. Gadepally, S. Samsi, and D. Tiwari, “Characterizing multi-instance GPU for machine learning workloads,” in *Proc. IEEE Int. Parallel and Distributed Processing Symp. Workshops (IPDPSW)*, 2022.
- [27] B. Li, T. Patel, S. Samsi, V. Gadepally, and D. Tiwari, “MISO: Exploiting multi-instance GPU capability on multi-tenant GPU clusters,” in *Proc. 13th ACM Symp. on Cloud Computing (SoCC)*, 2022.
- [28] K. Haghshenas and M. Hashemi, “EaCO: Resource sharing dynamics and its impact on energy efficiency for DNN training,” arXiv:2412.08294, 2024.
- [29] M. Andoorvedu, Z. Zhu, B. Zheng, and G. Pekhimenko, “Tempo: Accelerating transformer-based model training through memory footprint reduction,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [30] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefler, “Data movement is all you need: A case study on optimizing transformers,” in *Proc. Machine Learning and Systems (MLSys)*, 2021.
- [31] J. Mohan, A. Phanishayee, and V. Chidambaram, “CheckFreq: Frequent, fine-grained DNN checkpointing,” in *Proc. 19th USENIX Conf. on File and Storage Technologies (FAST)*, 2021.

- [32] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [33] J. You, J.-W. Chung, and M. Chowdhury, “Zeus: Understanding and optimizing GPU energy consumption of DNN training,” in *Proc. 20th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2023.
- [34] D. Zhao et al., “Sustainable supercomputing for AI: GPU power capping at HPC scale,” in *Proc. ACM Symp. on Cloud Computing (SoCC)*, 2023.
- [35] G. Koszczał, J. Dobrosolski, M. Matuszek, and P. Czarnul, “Performance and energy-aware training of a deep neural network in a multi-GPU environment with power capping,” in *Euro-Par 2023: Parallel Processing Workshops*, 2023.